

W01 — Manual da Demo

Enterprise RAG: DataOps Knowledge Hub | Walkthrough Completo

Pré-Requisitos

Antes de iniciar a demo, garantir:

- ☐ `docker compose ps` → todos os services healthy
 - ☐ `make ingest` executado → Qdrant populado
 - ☐ `make serve` rodando em background → API respondendo
 - ☐ MCP configurado no Claude Code → tools disponíveis
 - ☐ Segundo terminal aberto para logs
-

Parte 1 — Mostrar a Infra Viva

1.1 Docker Compose (2 min)

```
docker compose ps
```

Mostrar: todos os 6+ services com status “healthy”. Destacar os nomes: postgres, mongo, qdrant, neo4j, seaweedfs, data-generator.

1.2 Dados Crescendo em Tempo Real (3 min)

```
# Terminal 1: watch dos dados no Postgres
watch -n5 "docker compose exec -T postgres psql -U dataops -d dataops -c \
  \"SELECT 'customers' AS tbl, count(*) FROM customers UNION ALL \
  SELECT 'orders', count(*) FROM orders UNION ALL \
  SELECT 'products', count(*) FROM products;\""
```

Deixar rodando. Os números sobem a cada 30 segundos. Dizer:

“O sistema está vivo. Dados novos a cada 30 segundos. Isso simula uma operação real.”

1.3 MongoDB Events (1 min)

```
docker compose exec mongo mongosh dataops --eval \
  "print('event_logs:', db.event_logs.countDocuments()); \
  print('user_activity:', db.user_activity.countDocuments());"
```

Parte 2 — Mostrar Cada Data Store

2.1 PostgreSQL — O Ledger (3 min)

```
# Ver as tabelas
docker compose exec postgres psql -U dataops -d dataops -c "\dt"

# Ver dados reais
docker compose exec postgres psql -U dataops -d dataops -c \
  "SELECT name, email, plan, created_at FROM customers LIMIT 5;"

# Ver orders
docker compose exec postgres psql -U dataops -d dataops -c \
  "SELECT c.name, p.name AS product, o.quantity, o.amount, o.status \
  FROM orders o JOIN customers c ON o.customer_id = c.id \
  JOIN products p ON o.product_id = p.id \
  ORDER BY o.created_at DESC LIMIT 5;"
```

“Isso é o Ledger. Fatos. Números. A verdade transacional.”

2.2 Neo4j — O Brain (3 min)

Abrir no browser: `http://localhost:7474`

Login: `neo4j` / `dataops123`

Executar no Neo4j Browser:

```
// Ver todo o grafo (15 nodes, 17 relationships)
MATCH (n)-[r]->(m) RETURN n, r, m

// Ver dependências da tabela orders (quem lê, quem é dono, quais dashboards)
MATCH path = (t:Table {name: "orders"})-[*1..2]-(connected) RETURN path
```

“Isso é o Brain. Relacionamentos. Quem depende de quem. Se uma tabela cai, o que quebra?”

2.3 Qdrant — O Memory (2 min)

Abrir no browser: `http://localhost:6333/dashboard`

Mostrar:

- Collection `dataops_memory`
- Número de pontos indexados
- Clicar em um ponto → ver o metadata (title, summary, keywords, questions_answered)

“Isso é o Memory. Documentos transformados em vetores. Cada chunk tem metadata rico — título, resumo, keywords. Isso é o que torna o retrieval preciso.”

2.4 SeaweedFS — O Data Lake (1 min)

```
# Listar arquivos no bucket
curl -s http://localhost:8333/dataops-lake/ | python3 -c "import sys,
xml.etree.ElementTree as ET; root = ET.fromstring(sys.stdin.read());
[print(key.text) for key in root.iter('{http://s3.amazonaws.com/doc/2006-03-
01/}Key')]"
```

Output esperado:

```
data-dictionary.csv
data-retention-policy.md
incident-response-runbook.md
reports/daily-summary-2026-05-23.csv
sla-definitions.md
```

UI do SeaweedFS:

- Master Server: `http://localhost:9333` (status do cluster, volumes)
- Ver documento direto no browser: `http://localhost:8333/dataops-lake/data-retention-policy.md`

“Aqui estão os documentos originais: políticas, SLAs, runbooks, e até relatórios diários gerados automaticamente. Posso abrir qualquer um direto no browser. O pipeline de

ingestion leu, chunkeou, enriqueceu e indexou tudo no Qdrant.”

Parte 3 — Swagger UI (FastAPI)

3.1 Abrir a Documentação (2 min)

Abrir no browser: `http://localhost:8000/docs`

Mostrar:

- Os endpoints documentados automaticamente
- O schema do request (QueryRequest)
- O schema do response (QueryResponse)
- O health check

“Isso é FastAPI + Pydantic. Zero esforço de documentação. O schema Pydantic vira Swagger automaticamente.”

3.2 Testar via Swagger (3 min)

Clicar em `POST /api/v1/query` → Try it out:

```
{  
  "question": "Quantos clientes enterprise nós temos?"  
}
```

Mostrar o response completo: answer, sources_consulted, confidence, processing_time_ms.

Parte 4 — Testar Cada Engine via curl

4.1 Ledger — Text-to-SQL (2 min)

```
curl -s -X POST http://localhost:8000/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Quais são os top 5 clientes do plano enterprise por receita total?", "sources": ["ledger"]}' \
  | python3 -m json.tool
```

Destacar: `sql_query_executed` no response. O LLM gerou SQL real.

“Perguntei em português. O sistema gerou SQL, executou no Postgres, e me deu a resposta estruturada.”

4.2 Memory — Vector Search (2 min)

```
curl -s -X POST http://localhost:8000/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Qual é a política de retenção de dados PII?", "sources": ["memory"]}' \
  | python3 -m json.tool
```

Destacar: `sources` com os chunks recuperados e confidence score.

“Busca semântica nos documentos. Perguntei em português e ele encontrou a política de retenção com as regras exatas.”

4.3 Brain — Graph Traversal (2 min)

```
curl -s -X POST http://localhost:8000/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Quem é o owner do pipeline etl_billing_daily e quais tabelas ele lê?", "sources": ["brain"]}' \
  | python3 -m json.tool
```

Destacar: `cypher_query_executed` e `nodes_traversed`.

“Gerou Cypher, traversou o grafo, e me disse exatamente quem é dono do pipeline e quais tabelas ele lê.”

Parte 5 — Cross-Domain Query (O Momento WOW)

5.1 A Pergunta que Pega os 3 (5 min)

```
curl -s -X POST http://localhost:8000/api/v1/query \
  -H "Content-Type: application/json" \
  -d '{"question": "Quais são os top clientes por faturamento, qual o SLA do pipeline de billing, e qual seria o impacto se a tabela orders ficasse indisponível?"}' \
  | python3 -m json.tool
```

Pausar aqui. Deixar o response carregar (pode levar 10-20s).

Mostrar no response:

- `sub_questions` : 3 perguntas decompostas
- `sources_consulted` : 3 sources (ledger, memory, brain)
- Cada source com `query_used` (SQL, vector search, Cypher)
- `synthesized_answer` : resposta combinada
- `recommendation` : ação sugerida
- `confidence` : score geral
- `processing_time_ms` : tempo total

“UMA pergunta em português. O sistema SOZINHO decidiu que precisava de 3 fontes. Gerou SQL para o Postgres, buscou no Qdrant, traversou o Neo4j. E sintetizou tudo em uma resposta coerente com recomendação. Isso é RAG enterprise.”

5.2 Mostrar os Logs (1 min)

No terminal de logs:

```
docker compose logs app --tail 20
```

Mostrar: o request chegando, os 3 engines sendo chamados, o tempo de cada um.

Parte 6 — Grand Finale: Claude Code + MCP

6.0 Pré-requisito: CLAUDE.md com instrução MCP (1 min)

IMPORTANTE: Para o Claude Code usar o MCP de forma transparente (sem precisar forçar), adicionar no `CLAUDE.md` do projeto:

MCP Tools Available

This project exposes a local MCP server (``dataops-knowledge-hub``). When asked business questions about customers, pipelines, SLAs, revenue, data quality, or infrastructure impact – ALWAYS use the ``query_knowledge_hub`` MCP tool instead of trying to answer from code or documentation.

Available MCP tools:

- ``query_knowledge_hub`` – Ask any business question (routes to Postgres/Qdrant/Neo4j)
- ``check_platform_health`` – Check status of all data stores
- ``trigger_ingestion`` – Re-index documents from SeaweedFS and MongoDB

Isso garante que o Claude Code entende que perguntas de negócio devem ir para o MCP, não para o código.

Configurar o MCP no Claude Code (se ainda não fez):

```
claude mcp add dataops-knowledge-hub -- python3 -m src.mcp.run
```

6.1 Setup (2 min)

Abrir um **NOVO terminal**. Iniciar o Claude Code:

claude

Verificar que o MCP está configurado:

“Quais tools você tem disponíveis?”

Claude deve listar: `query_knowledge_hub`, `check_platform_health`, `trigger_ingestion`.

6.2 Perguntas Progressivas (10 min)

Pergunta 1 — Simple (Ledger):

Quais são os top 5 clientes do plano enterprise por receita total?

Esperar. Claude descobre a tool → chama → responde.

Pergunta 2 — Memory:

Qual é a política de retenção de dados PII da empresa? Quanto tempo podemos manter?

Pergunta 3 — Brain:

Quem é o owner do pipeline `etl_billing_daily` e quais tabelas ele lê?

Pergunta 4 — Cross-Domain (GRAND FINALE):

Me dê um relatório executivo: quais são os top 5 clientes por faturamento, qual o SLA do pipeline de billing, e qual seria o impacto operacional se a tabela `orders` ficasse indisponível? Inclua recomendações.

Esperar. Mostrar o Claude Code chamando a tool, recebendo o JSON, e formatando a resposta.

6.3 Encerramento (2 min)

“O que vocês acabaram de ver:

O Claude Code — o mesmo agente que CONSTRUIU esse sistema — agora CONSOME ele via MCP.

Ele não sabe SQL. Não sabe Cypher. Não sabe onde os documentos estão. Mas ele tem uma TOOL que sabe. E essa tool é o sistema que nós construímos do zero nas últimas horas.

Isso é o loop completo do AI-Native Engineer. Vocês orquestram agentes que constroem sistemas que são consumidos por agentes.

No próximo workshop, o Rafael vai pegar essa API e construir um agente autônomo com LangGraph que monitora a qualidade dos dados ²⁴/₇. O que vocês construíram hoje é a fundação.”

Timing Total da Demo

Parte	Duração	Acumulado
1. Infra Viva	6 min	6 min
2. Cada Data Store	9 min	15 min
3. Swagger UI	5 min	20 min
4. Engines via curl	6 min	26 min
5. Cross-Domain (WOW)	6 min	32 min
6. Grand Finale (MCP)	14 min	46 min

Total: ~46 minutos de demo pura.

Contingências

Problema	Solução
Engine demora > 30s	“A OpenAI está com latência. Olhem o que já retornou...” (mostrar partial)
MCP não conecta	Fazer via curl direto. Menos impactante mas funciona.
Neo4j lento	Já ter o Cypher result em um screenshot. Mostrar e seguir.
Qdrant vazio	Rodar <code>make ingest</code> ao vivo. “Vejam o pipeline rodando.”
Docker crash	<code>docker compose restart <service></code> . Continuar com os outros.